School of Computer Science and Artificial Intelligence

Lab Assignment # 10

Course Title : AI Assistant Coding
Name of Student : BG Sreevani
Enrollment No. : 2303A54066
Batch No. : 48

Lab 10 – Code Review and Quality: Using AI to improve code quality and readability

Problem Statement 1: AI-Assisted Bug Detection

Scenario: A junior developer wrote the following Python function to calculate factorials:

```
def factorial(n):
result = 1
for i in range(1, n):
result = result * i
return result
```

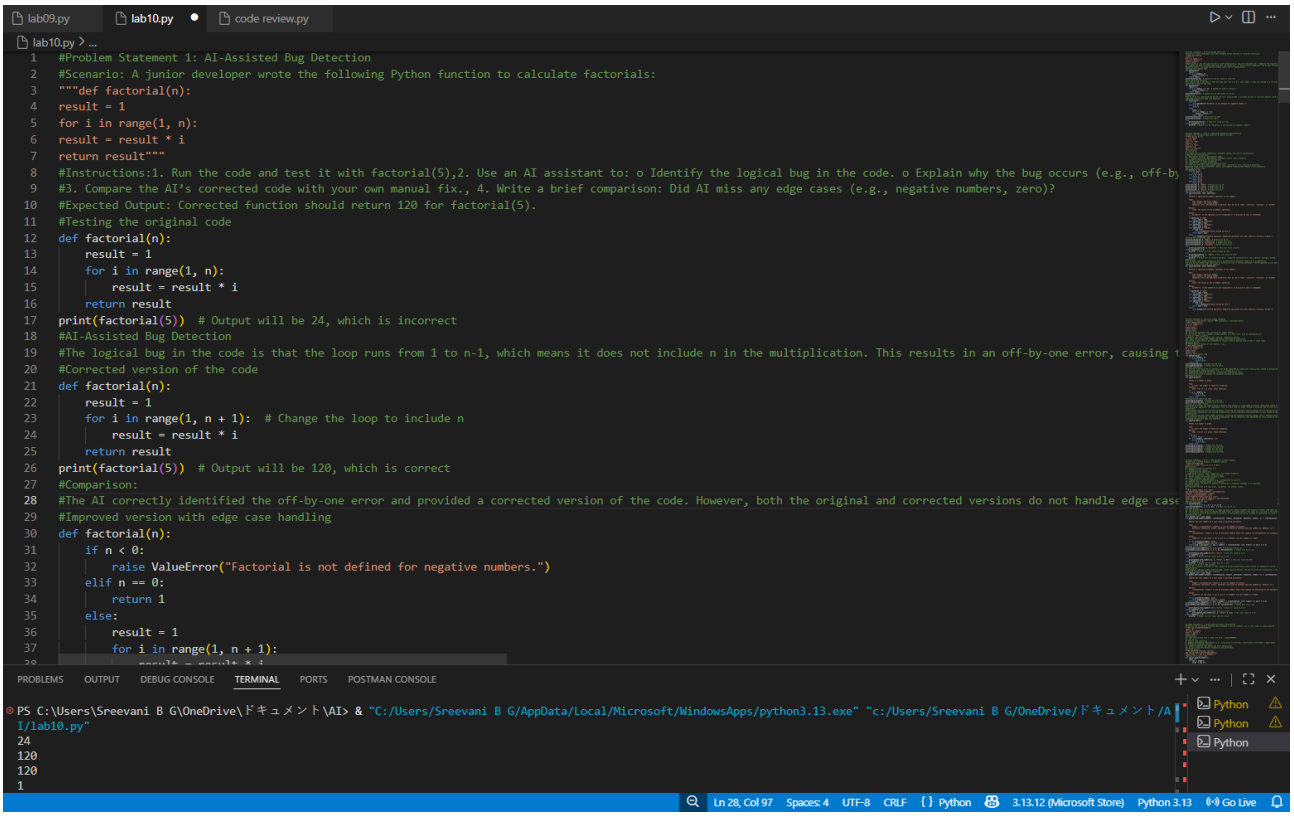
Instructions:

1. Run the code and test it with factorial(5).
2. Use an AI assistant to:
   - o Identify the logical bug in the code.
   - o Explain why the bug occurs (e.g., off-by-one error).
   - o Provide a corrected version.
3. Compare the AI's corrected code with your own manual fix.
4. Write a brief comparison: Did AI miss any edge cases (e.g.,negative numbers, zero)?

Expected Output:

Corrected function should return 120 for factorial(5).

• **Screenshot of Generated Code:**



Problem Statement 2: Improving Readability & Documentation

Scenario:The following code works but is poorly written:

```
def calc(a, b, c):
if c == "add":
return a + b
elif c == "sub":
return a - b
elif c == "mul":
return a * b
elif c == "div":
```

Instructions:

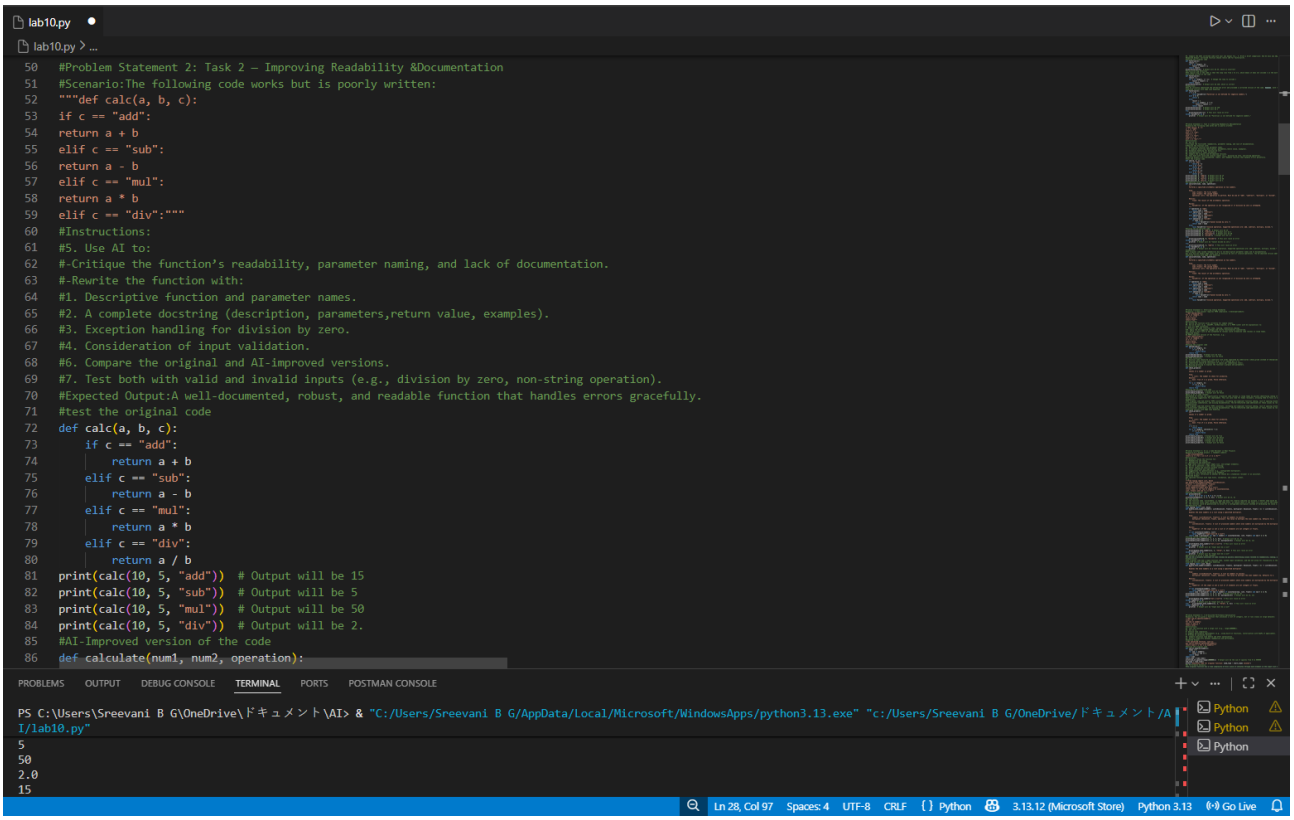
5. Use AI to:

- **o Critique the function's readability, parameter naming, and lack of documentation.**
- **o Rewrite the function with:**

1. Descriptive function and parameter names.
2. A complete docstring (description, parameters,return value, examples).
3. Exception handling for division by zero.
4. Consideration of input validation.
6. Compare the original and AI-improved versions.
7. Test both with valid and invalid inputs (e.g., division by zero, non-string operation).

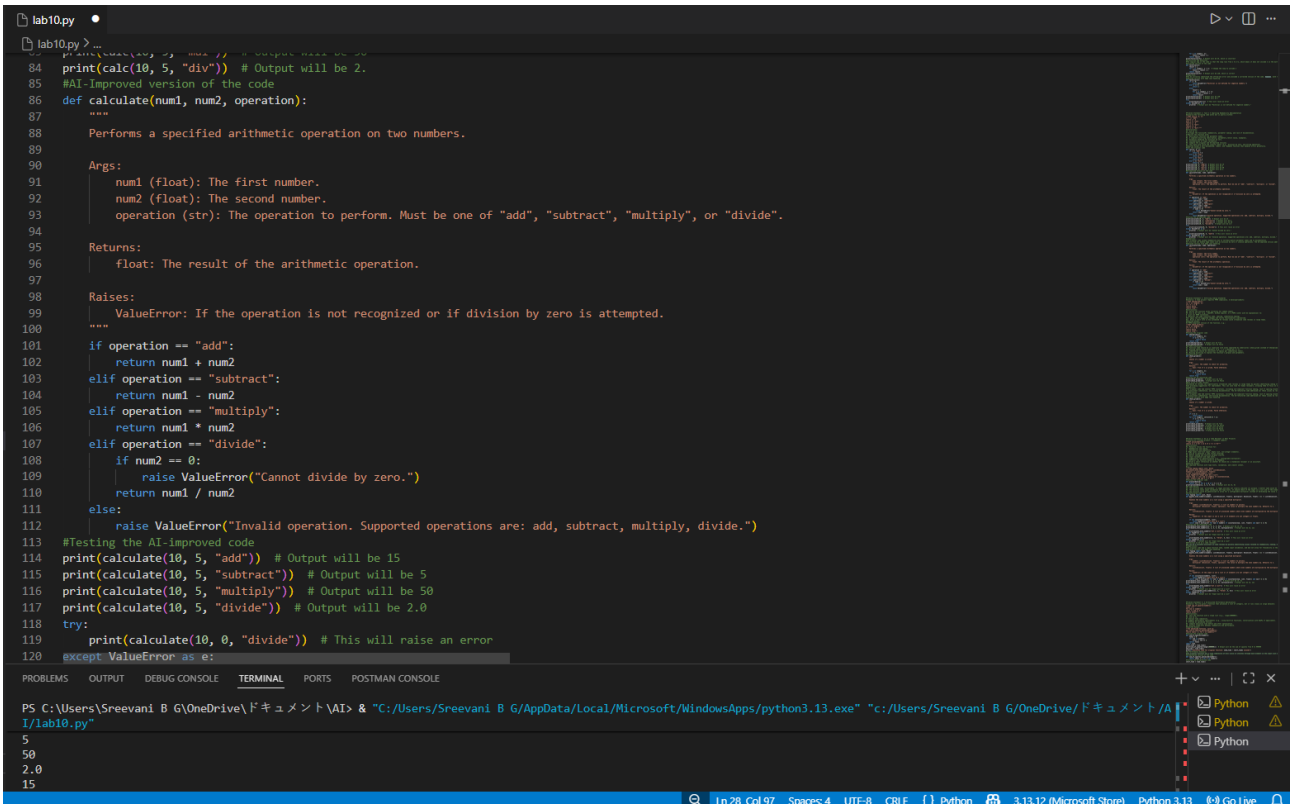
Expected Output:A well-documented, robust, and readable function that handles errors gracefully.

• Screenshot of Generated Code:



```
#Problem Statement 2: Task 2 – Improving Readability &Documentation
#Scenario:The following code works but is poorly written:
"""def calc(a, b, c):
if c == "add":
return a + b
elif c == "sub":
return a - b
elif c == "mul":
return a * b
elif c == "div":"""
#Instructions:
#5. Use AI to:
#-Critique the function's readability, parameter naming, and lack of documentation.
#-Rewrite the function with:
#1. Descriptive function and parameter names.
#2. A complete docstring (description, parameters,return value, examples).
#3. Exception handling for division by zero.
#4. Consideration of input validation.
#6. Compare the original and AI-improved versions.
#7. Test both with valid and invalid inputs (e.g., division by zero, non-string operation).
#Expected Output:A well-documented, robust, and readable function that handles errors gracefully.
#test the original code
def calc(a, b, c):
    if c == "add":
        return a + b
    elif c == "sub":
        return a - b
    elif c == "mul":
        return a * b
    elif c == "div":
        return a / b
print(calc(10, 5, "add"))  # Output will be 15
print(calc(10, 5, "sub"))  # Output will be 5
print(calc(10, 5, "mul"))  # Output will be 50
print(calc(10, 5, "div"))  # Output will be 2.
#AI-Improved version of the code
def calculate(num1, num2, operation):
```

```
PS C:\Users\Sreevani B G\OneDrive\ドキュメント\AI> & "C:/Users/Sreevani B G/AppData/Local/Microsoft/WindowsApps/python3.13.exe" "c:/Users/Sreevani B G/OneDrive/ドキュメント/AI/lab10.py"
5
50
2.0
15
```



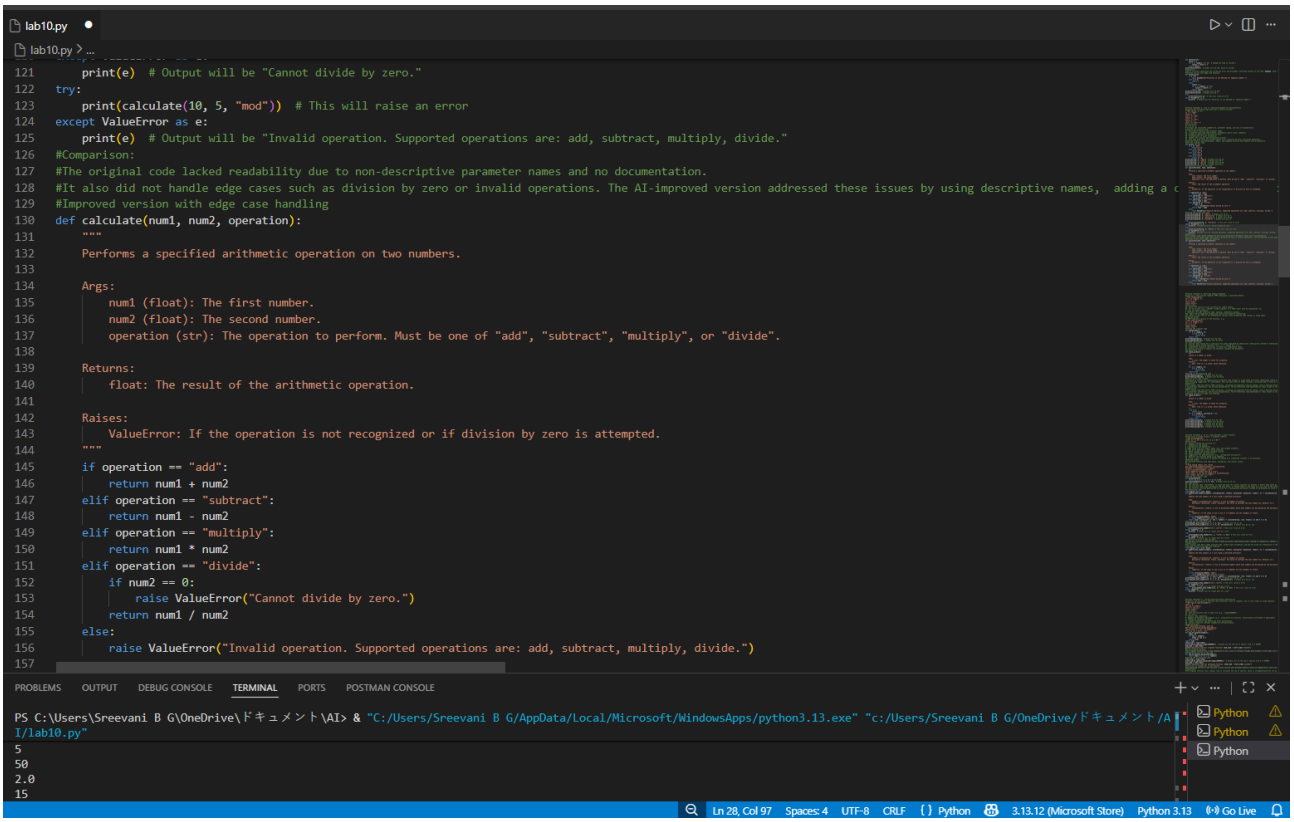
```
print(calc(10, 5, "div"))  # Output will be 2.
#AI-Improved version of the code
def calculate(num1, num2, operation):
    """
    Performs a specified arithmetic operation on two numbers.

    Args:
        num1 (float): The first number.
        num2 (float): The second number.
        operation (str): The operation to perform. Must be one of "add", "subtract", "multiply", or "divide".

    Returns:
        float: The result of the arithmetic operation.

    Raises:
        ValueError: If the operation is not recognized or if division by zero is attempted.
    """
    if operation == "add":
        return num1 + num2
    elif operation == "subtract":
        return num1 - num2
    elif operation == "multiply":
        return num1 * num2
    elif operation == "divide":
        if num2 == 0:
            raise ValueError("Cannot divide by zero.")
        return num1 / num2
    else:
        raise ValueError("Invalid operation. Supported operations are: add, subtract, multiply, divide.")
#Testing the AI-improved code
print(calculate(10, 5, "add"))  # Output will be 15
print(calculate(10, 5, "subtract"))  # Output will be 5
print(calculate(10, 5, "multiply"))  # Output will be 50
print(calculate(10, 5, "divide"))  # Output will be 2.0
try:
    print(calculate(10, 0, "divide"))  # This will raise an error
except ValueError as e:
```

```
PS C:\Users\Sreevani B G\OneDrive\ドキュメント\AI> & "C:/Users/Sreevani B G/AppData/Local/Microsoft/WindowsApps/python3.13.exe" "c:/Users/Sreevani B G/OneDrive/ドキュメント/AI/lab10.py"
5
50
2.0
15
```



Problem Statement 3: Enforcing Coding Standards

Scenario: A team project requires PEP8 compliance. A developersubmits:

```
def Checkprime(n):
for i in range(2, n):
if n % i == 0:
return False
return True
```

Instructions:

8. Verify the function works correctly for sample inputs.
9. Use an AI tool (e.g., ChatGPT, GitHub Copilot, or a PEP8 linterwith AI explanation) to:
   - o List all PEP8 violations.
   - o Refactor the code (function name, spacing, indentation, naming).
10. Apply the AI-suggested changes and verify functionality ispreserved.
11. Write a short note on how automated AI reviews could streamline code reviews in large teams.

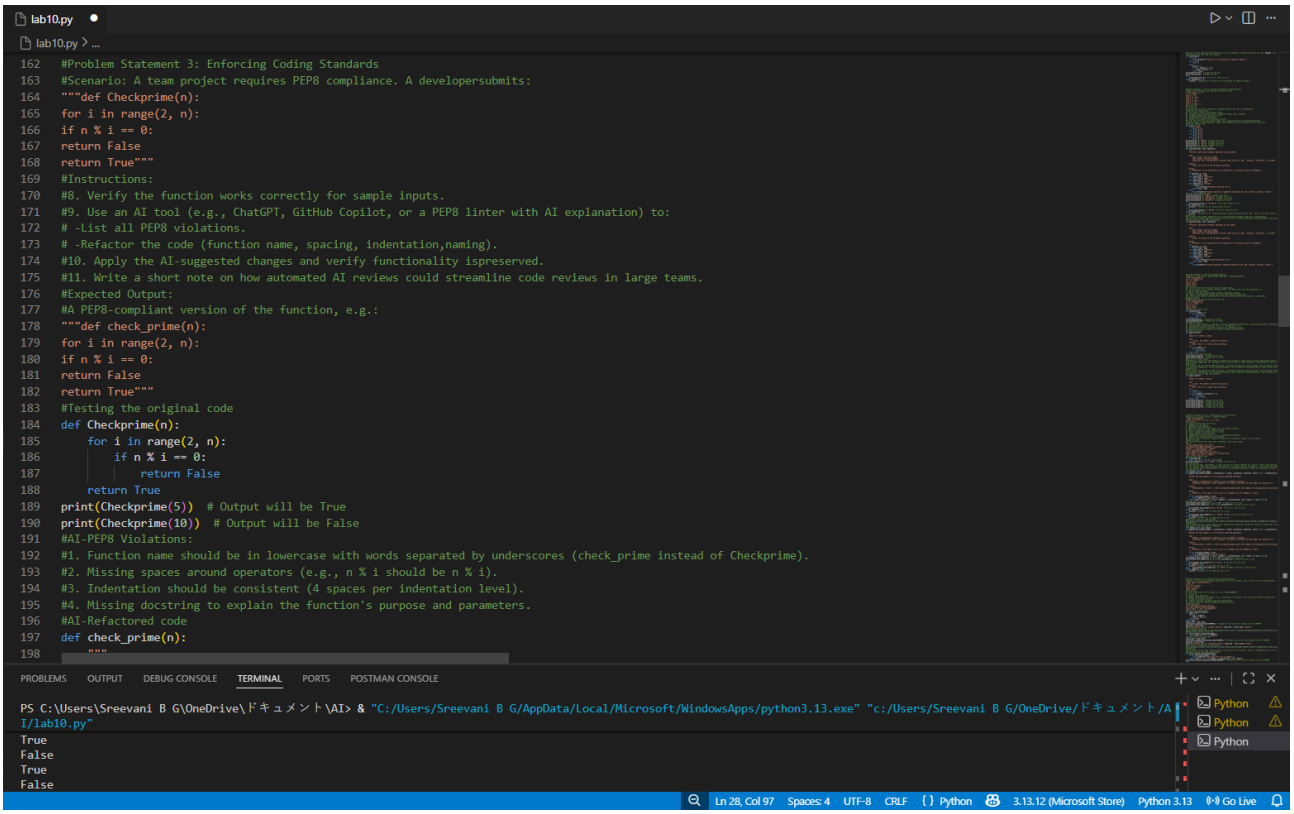
Expected Output:

A PEP8-compliant version of the function, e.g.:

```
def check_prime(n):
for i in range(2, n):
if n % i == 0:
return False
return True
```

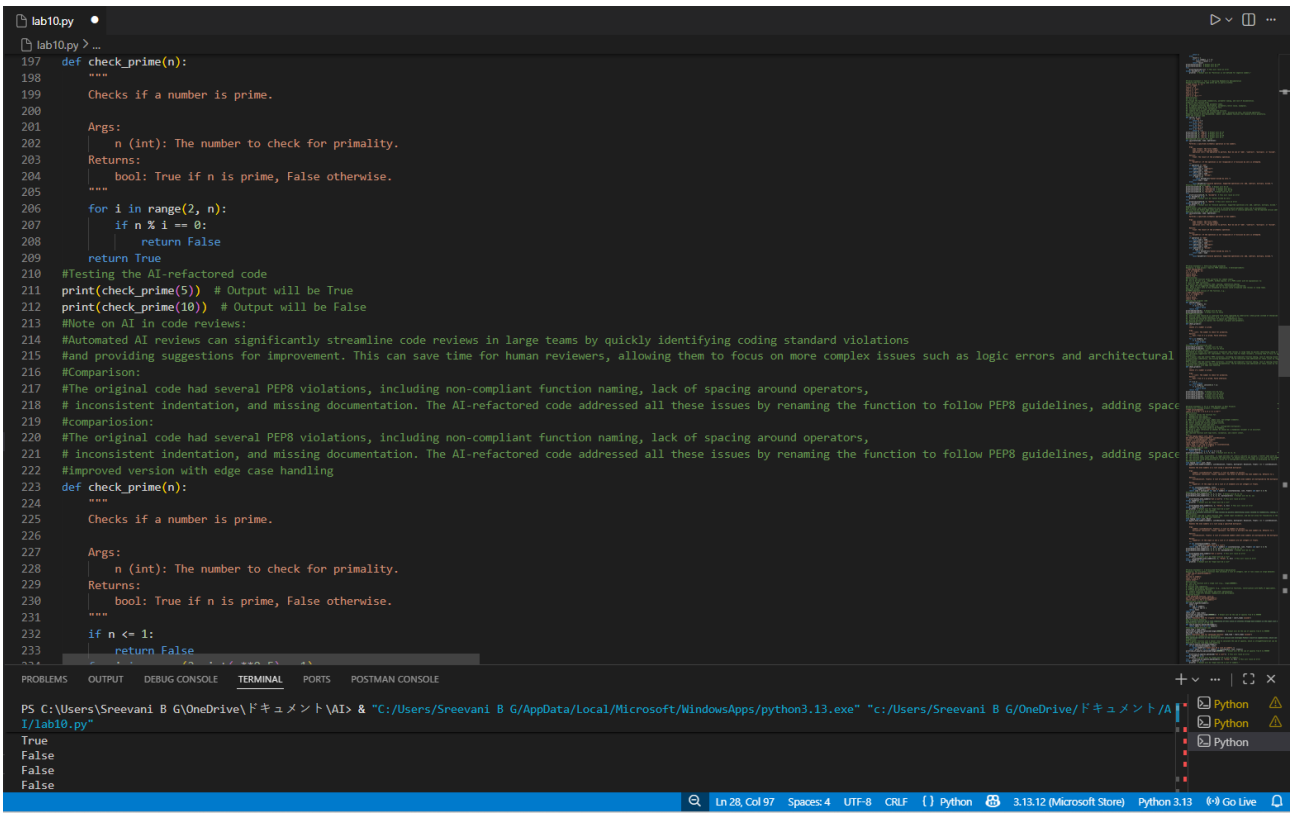
- **Screenshot of Generated Code(V1):**

- **Screenshot of Generated Code(V2):**



```
#Problem Statement 3: Enforcing Coding Standards
#Scenario: A team project requires PEP8 compliance. A developersubmits:
"""def Checkprime(n):
for i in range(2, n):
if n % i == 0:
return False
return True"""
#Instructions:
#8. Verify the function works correctly for sample inputs.
#9. Use an AI tool (e.g., ChatGPT, GitHub Copilot, or a PEP8 linter with AI explanation) to:
# -List all PEP8 violations.
# -Refactor the code (function name, spacing, indentation,naming).
#10. Apply the AI-suggested changes and verify functionality ispreserved.
#11. Write a short note on how automated AI reviews could streamline code reviews in large teams.
#Expected Output:
#A PEP8-compliant version of the function, e.g.:
"""def check_prime(n):
for i in range(2, n):
if n % i == 0:
return False
return True"""
#Testing the original code
def Checkprime(n):
    for i in range(2, n):
        if n % i == 0:
            return False
    return True
print(Checkprime(5))  # Output will be True
print(Checkprime(10))  # Output will be False
#AI-PEP8 Violations:
#1. Function name should be in lowercase with words separated by underscores (check_prime instead of Checkprime).
#2. Missing spaces around operators (e.g., n % i should be n % i).
#3. Indentation should be consistent (4 spaces per indentation level).
#4. Missing docstring to explain the function's purpose and parameters.
#AI-Refactored code
def check_prime(n):
    """
```

```
PS C:\Users\Sreevani B G\OneDrive\ドキュメント\AI> & "C:/Users/Sreevani B G/AppData/Local/Microsoft/WindowsApps/python3.13.exe" "c:/Users/Sreevani B G/OneDrive/ドキュメント/AI/lab10.py"
True
False
True
False
```

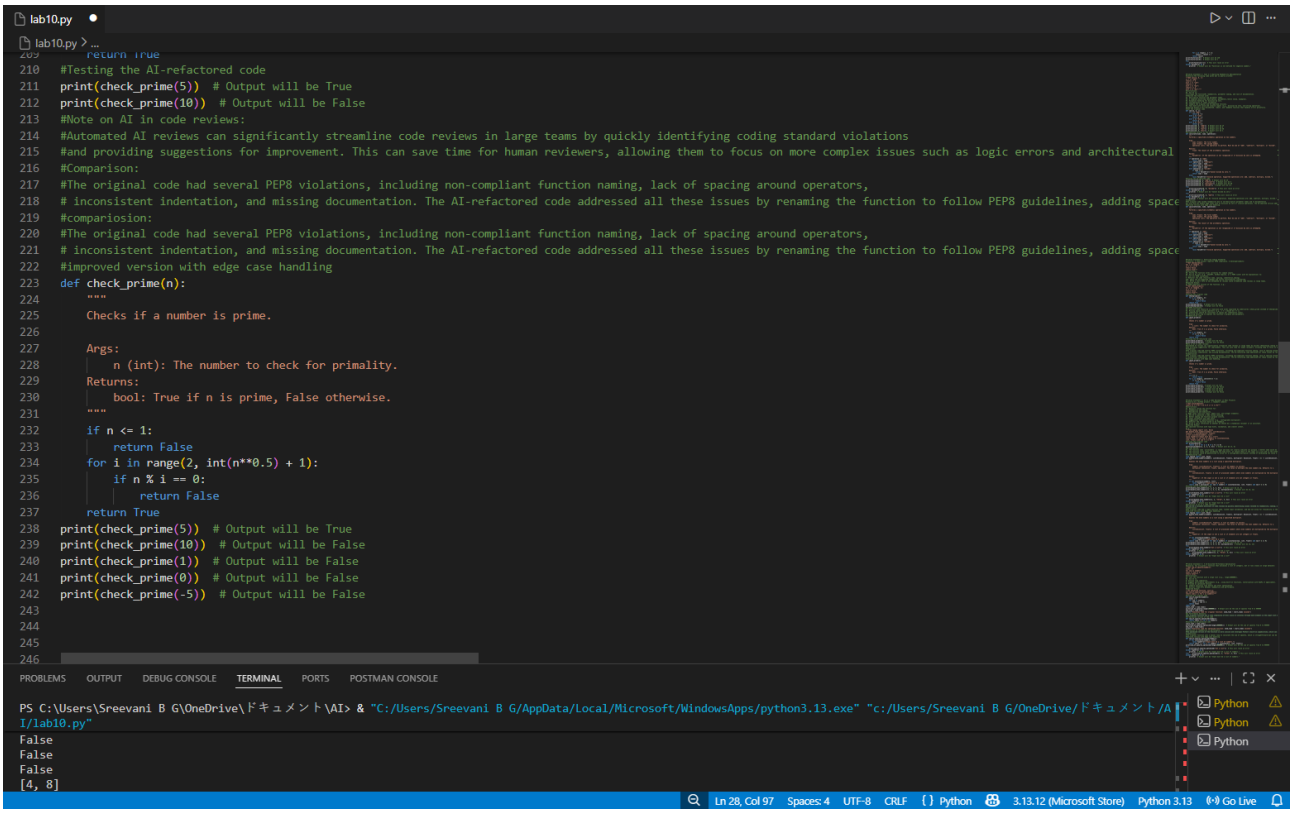


```
def check_prime(n):
    """
    Checks if a number is prime.

    Args:
        n (int): The number to check for primality.
    Returns:
        bool: True if n is prime, False otherwise.
    """
    for i in range(2, n):
        if n % i == 0:
            return False
    return True
#Testing the AI-refactored code
print(check_prime(5))  # Output will be True
print(check_prime(10))  # Output will be False
#Note on AI in code reviews:
#Automated AI reviews can significantly streamline code reviews in large teams by quickly identifying coding standard violations
#and providing suggestions for improvement. This can save time for human reviewers, allowing them to focus on more complex issues such as logic errors and architectural
#Comparison:
#The original code had several PEP8 violations, including non-compliant function naming, lack of spacing around operators,
# inconsistent indentation, and missing documentation. The AI-refactored code addressed all these issues by renaming the function to follow PEP8 guidelines, adding space
#compariosion:
#The original code had several PEP8 violations, including non-compliant function naming, lack of spacing around operators,
# inconsistent indentation, and missing documentation. The AI-refactored code addressed all these issues by renaming the function to follow PEP8 guidelines, adding space
#improved version with edge case handling
def check_prime(n):
    """
    Checks if a number is prime.

    Args:
        n (int): The number to check for primality.
    Returns:
        bool: True if n is prime, False otherwise.
    """
    if n <= 1:
        return False
```

```
PS C:\Users\Sreevani B G\OneDrive\ドキュメント\AI> & "C:/Users/Sreevani B G/AppData/Local/Microsoft/WindowsApps/python3.13.exe" "c:/Users/Sreevani B G/OneDrive/ドキュメント/AI/lab10.py"
True
False
False
False
```



Problem Statement 4: AI as a Code Reviewer in Real Projects

Scenario:In a GitHub project, a teammate submits:

```
def processData(d):
return [x * 2 for x in d if x % 2 == 0]
```

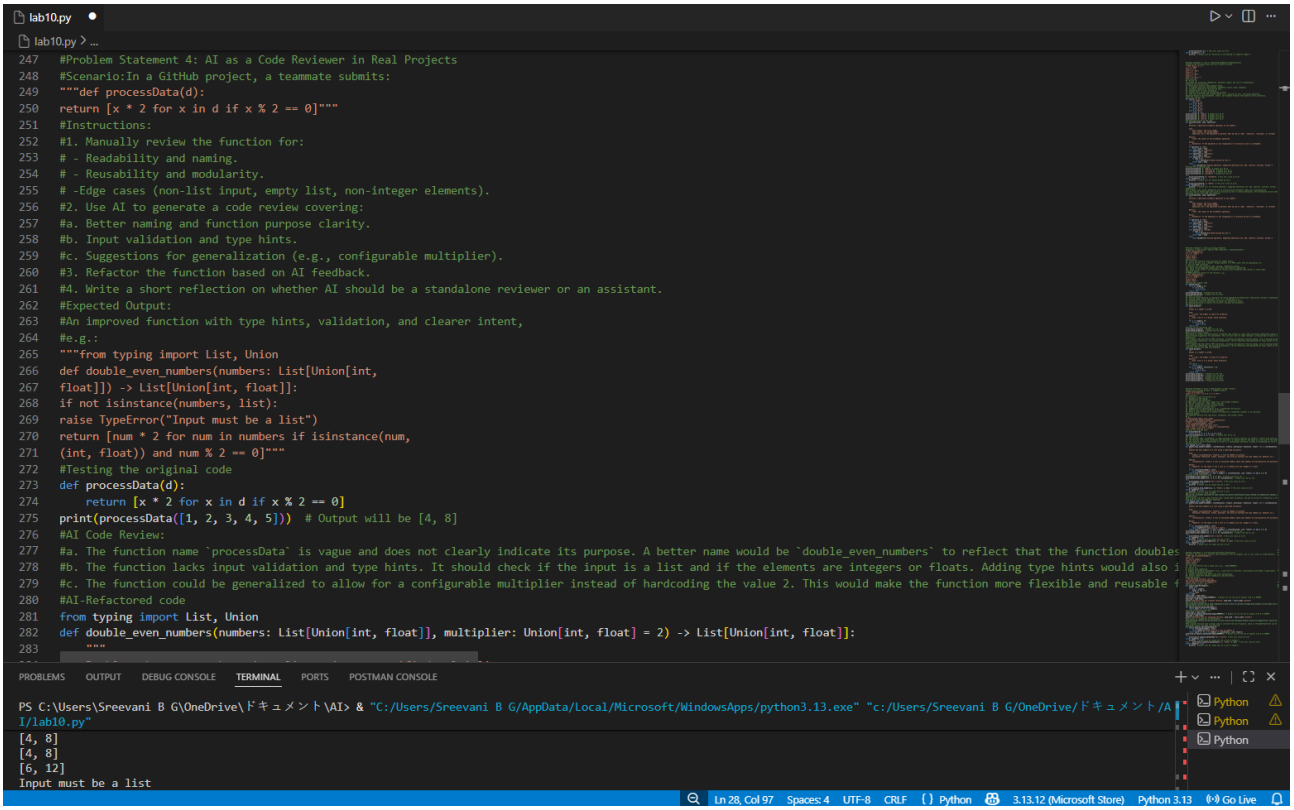
Instructions:

1. Manually review the function for:
   - o Readability and naming.
   - o Reusability and modularity.
   - o Edge cases (non-list input, empty list, non-integerelements).
2. Use AI to generate a code review covering:
   - a. Better naming and function purpose clarity.
   - b. Input validation and type hints.
   - c. Suggestions for generalization (e.g., configurable multiplier).
3. Refactor the function based on AI feedback.
4. Write a short reflection on whether AI should be a standalone reviewer or an assistant.

Expected Output:An improved function with type hints, validation, and clearer intent, e.g.:

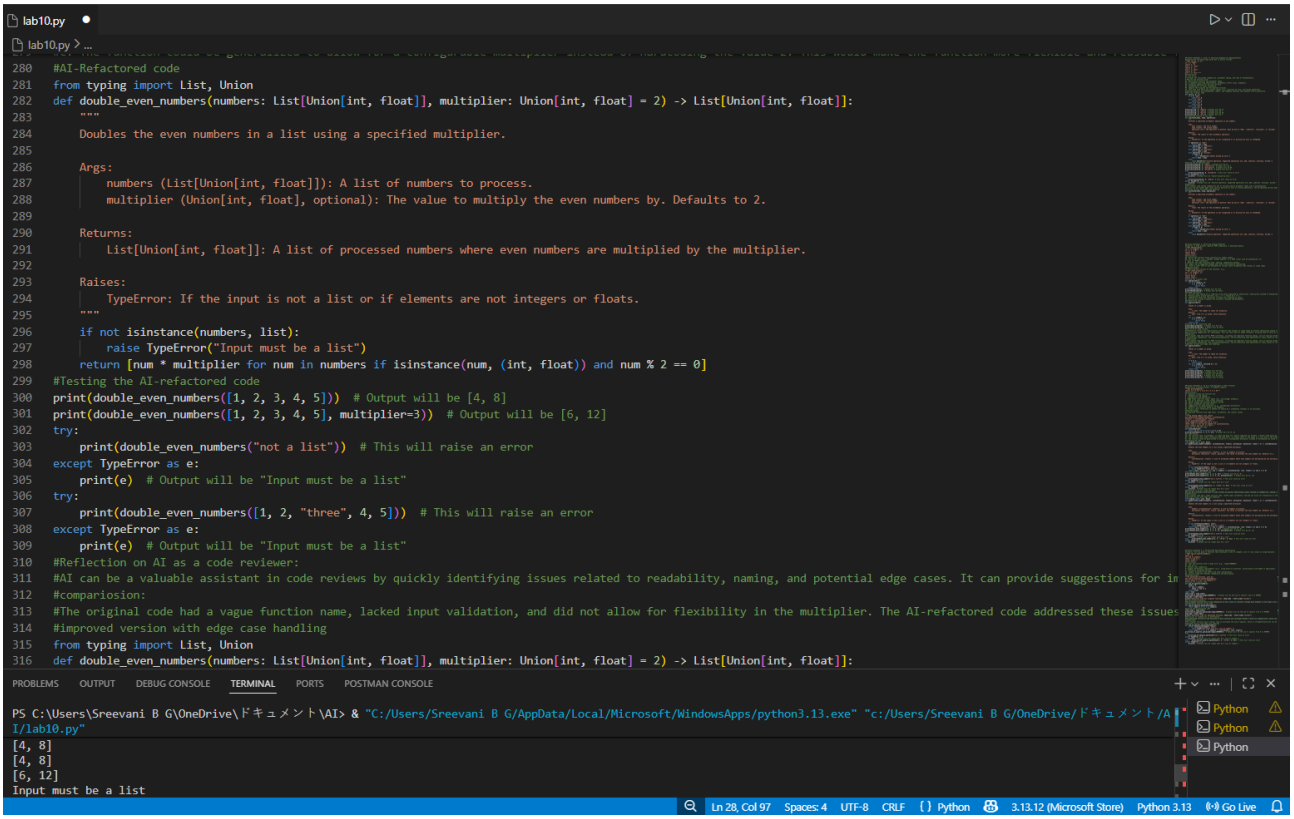
```
from typing import List, Union
def double_even_numbers(numbers: List[Union[int, float]]) -> List[Union[int, float]]:
if not isinstance(numbers, list):
raise TypeError("Input must be a list")
return [num * 2 for num in numbers if isinstance(num, (int, float)) and num % 2 == 0]
```

• Screenshot of Generated Code



```
#Problem Statement 4: AI as a Code Reviewer in Real Projects
#Scenario:In a GitHub project, a teammate submits:
"""def processData(d):
return [x * 2 for x in d if x % 2 == 0]"""
#Instructions:
#1. Manually review the function for:
# - Readability and naming.
# - Reusability and modularity.
# -Edge cases (non-list input, empty list, non-integer elements).
#2. Use AI to generate a code review covering:
#a. Better naming and function purpose clarity.
#b. Input validation and type hints.
#c. Suggestions for generalization (e.g., configurable multiplier).
#3. Refactor the function based on AI feedback.
#4. Write a short reflection on whether AI should be a standalone reviewer or an assistant.
#Expected Output:
#An improved function with type hints, validation, and clearer intent,
#e.g.:
"""from typing import List, Union
def double_even_numbers(numbers: List[Union[int,
float]]) -> List[Union[int, float]]:
if not isinstance(numbers, list):
raise TypeError("Input must be a list")
return [num * 2 for num in numbers if isinstance(num,
(int, float)) and num % 2 == 0]"""
#Testing the original code
def processData(d):
    return [x * 2 for x in d if x % 2 == 0]
print(processData([1, 2, 3, 4, 5]))  # Output will be [4, 8]
#AI Code Review:
#a. The function name `processData` is vague and does not clearly indicate its purpose. A better name would be `double_even_numbers` to reflect that the function doubles
#b. The function lacks input validation and type hints. It should check if the input is a list and if the elements are integers or floats. Adding type hints would also i
#c. The function could be generalized to allow for a configurable multiplier instead of hardcoding the value 2. This would make the function more flexible and reusable f
#AI-Refactored code
from typing import List, Union
def double_even_numbers(numbers: List[Union[int, float]], multiplier: Union[int, float] = 2) -> List[Union[int, float]]:
    """
```

```
PS C:\Users\Sreevani B G\OneDrive\ドキュメント\AI> & "C:/Users/Sreevani B G/AppData/Local/Microsoft/WindowsApps/python3.13.exe" "c:/Users/Sreevani B G/OneDrive/ドキュメント/AI/lab10.py"
[4, 8]
[4, 8]
[6, 12]
Input must be a list
```



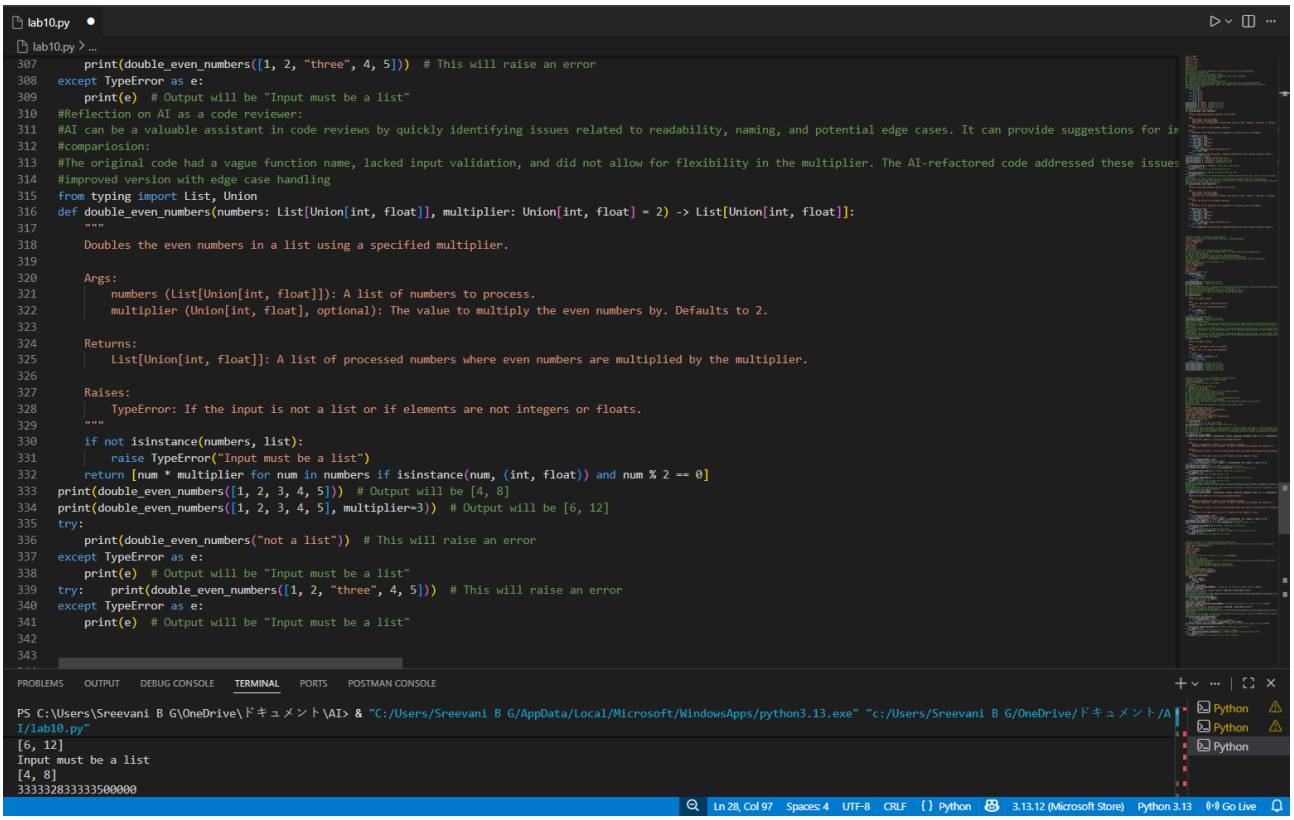
```
#AI-Refactored code
from typing import List, Union
def double_even_numbers(numbers: List[Union[int, float]], multiplier: Union[int, float] = 2) -> List[Union[int, float]]:
    """
    Doubles the even numbers in a list using a specified multiplier.

    Args:
        numbers (List[Union[int, float]]): A list of numbers to process.
        multiplier (Union[int, float], optional): The value to multiply the even numbers by. Defaults to 2.

    Returns:
        List[Union[int, float]]: A list of processed numbers where even numbers are multiplied by the multiplier.

    Raises:
        TypeError: If the input is not a list or if elements are not integers or floats.
    """
    if not isinstance(numbers, list):
        raise TypeError("Input must be a list")
    return [num * multiplier for num in numbers if isinstance(num, (int, float)) and num % 2 == 0]
#Testing the AI-refactored code
print(double_even_numbers([1, 2, 3, 4, 5]))  # Output will be [4, 8]
print(double_even_numbers([1, 2, 3, 4, 5], multiplier=3))  # Output will be [6, 12]
try:
    print(double_even_numbers("not a list"))  # This will raise an error
except TypeError as e:
    print(e)  # Output will be "Input must be a list"
try:
    print(double_even_numbers([1, 2, "three", 4, 5]))  # This will raise an error
except TypeError as e:
    print(e)  # Output will be "Input must be a list"
#Reflection on AI as a code reviewer:
#AI can be a valuable assistant in code reviews by quickly identifying issues related to readability, naming, and potential edge cases. It can provide suggestions for im
#compariosion:
#The original code had a vague function name, lacked input validation, and did not allow for flexibility in the multiplier. The AI-refactored code addressed these issues
#improved version with edge case handling
from typing import List, Union
def double_even_numbers(numbers: List[Union[int, float]], multiplier: Union[int, float] = 2) -> List[Union[int, float]]:
```

```
PS C:\Users\Sreevani B G\OneDrive\ドキュメント\AI> & "C:/Users/Sreevani B G/AppData/Local/Microsoft/WindowsApps/python3.13.exe" "c:/Users/Sreevani B G/OneDrive/ドキュメント/AI/lab10.py"
[4, 8]
[4, 8]
[6, 12]
Input must be a list
```



Problem Statement 5: — AI-Assisted Performance Optimization

Scenario: You are given a function that processes a list of integers, but it runs slowly on large datasets:

```
def sum_of_squares(numbers):
total = 0
for num in numbers:
total += num ** 2
return total
```

Instructions:

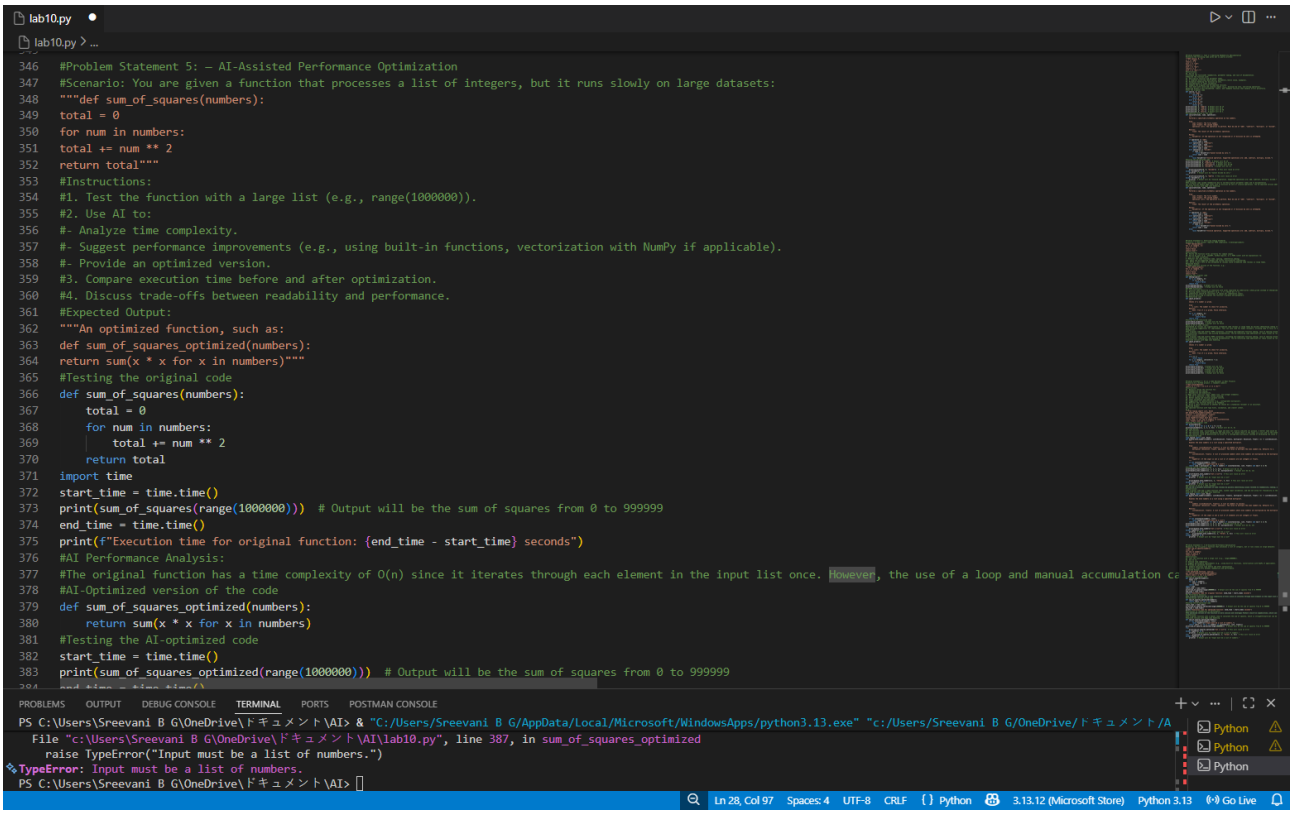
1. Test the function with a large list (e.g., range(1000000)).
2. Use AI to:
   - o Analyze time complexity.
   - o Suggest performance improvements (e.g., using built-in functions, vectorization with NumPy if applicable).
   - o Provide an optimized version.
3. Compare execution time before and after optimization.
4. Discuss trade-offs between readability and performance.

Expected Output:

An optimized function, such as:

```
def sum_of_squares_optimized(numbers):
return sum(x * x for x in numbers)
```

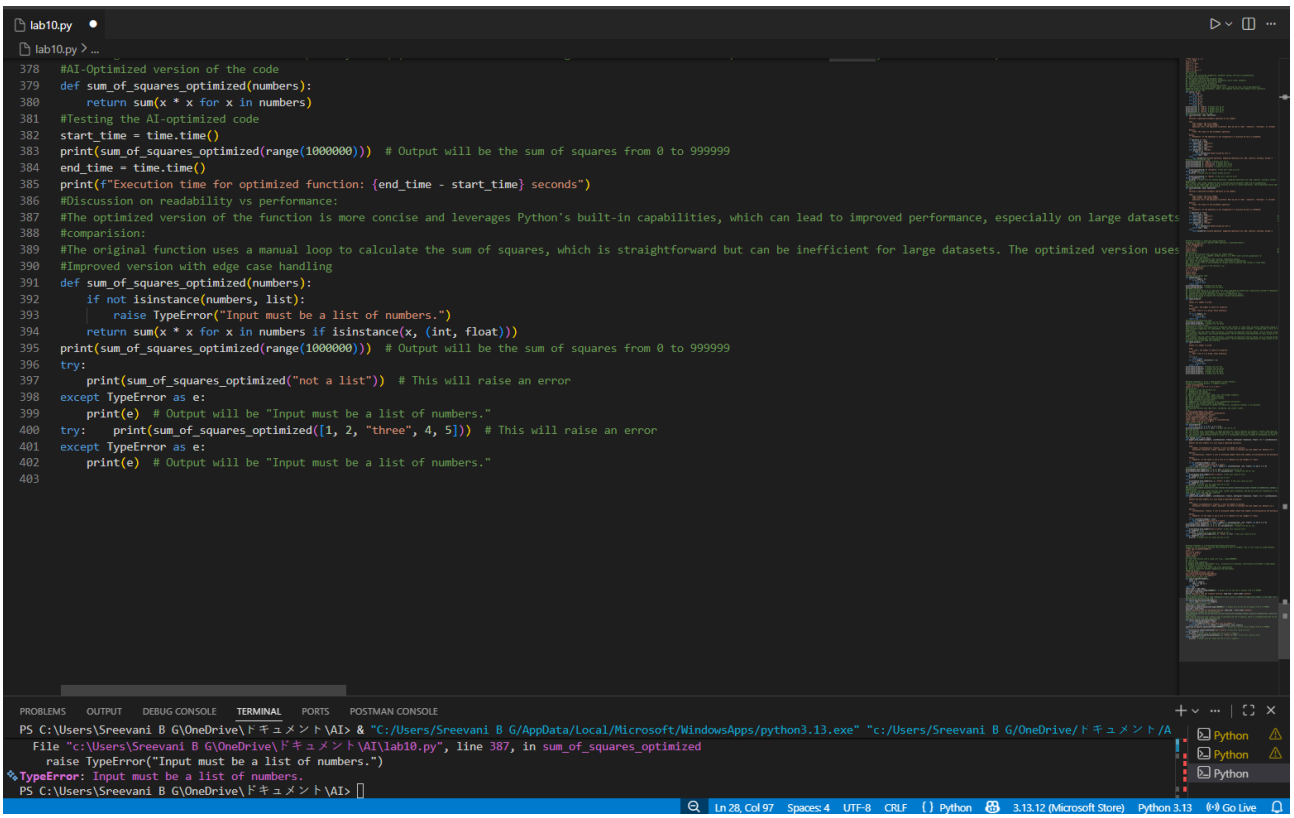
- **Screenshot of Generated Code**



```
lab10.py
#Problem Statement 5: – AI-Assisted Performance Optimization
#Scenario: You are given a function that processes a list of integers, but it runs slowly on large datasets:
"""def sum_of_squares(numbers):
total = 0
for num in numbers:
total += num ** 2
return total"""
#Instructions:
#1. Test the function with a large list (e.g., range(1000000)).
#2. Use AI to:
#- Analyze time complexity.
#- Suggest performance improvements (e.g., using built-in functions, vectorization with NumPy if applicable).
#- Provide an optimized version.
#3. Compare execution time before and after optimization.
#4. Discuss trade-offs between readability and performance.
#Expected Output:
"""An optimized function, such as:
def sum_of_squares_optimized(numbers):
return sum(x * x for x in numbers)"""
#Testing the original code
def sum_of_squares(numbers):
    total = 0
    for num in numbers:
        total += num ** 2
    return total
import time
start_time = time.time()
print(sum_of_squares(range(1000000)))  # Output will be the sum of squares from 0 to 999999
end_time = time.time()
print(f"Execution time for original function: {end_time - start_time} seconds")
#AI Performance Analysis:
#The original function has a time complexity of O(n) since it iterates through each element in the input list once. However, the use of a loop and manual accumulation ca
#AI-Optimized version of the code
def sum_of_squares_optimized(numbers):
    return sum(x * x for x in numbers)
#Testing the AI-optimized code
start_time = time.time()
print(sum_of_squares_optimized(range(1000000)))  # Output will be the sum of squares from 0 to 999999

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  POSTMAN CONSOLE
PS C:\Users\Sreevani B G\OneDrive\ドキュメント\AI> & "C:/Users/Sreevani B G/AppData/Local/Microsoft/WindowsApps/python3.13.exe" "c:/Users/Sreevani B G/OneDrive/ドキュメント/A
  File "c:\Users\Sreevani B G\OneDrive\ドキュメント\AI\lab10.py", line 387, in sum_of_squares_optimized
    raise TypeError("Input must be a list of numbers.")
TypeError: Input must be a list of numbers.
PS C:\Users\Sreevani B G\OneDrive\ドキュメント\AI>

Ln 28, Col 97  Spaces: 4  UTF-8  CRLF  { } Python  3.13.12 (Microsoft Store)  Python 3.13  Go Live
```



```
lab10.py
#AI-Optimized version of the code
def sum_of_squares_optimized(numbers):
    return sum(x * x for x in numbers)
#Testing the AI-optimized code
start_time = time.time()
print(sum_of_squares_optimized(range(1000000)))  # Output will be the sum of squares from 0 to 999999
end_time = time.time()
print(f"Execution time for optimized function: {end_time - start_time} seconds")
#Discussion on readability vs performance:
#The optimized version of the function is more concise and leverages Python's built-in capabilities, which can lead to improved performance, especially on large datasets
#comparision:
#The original function uses a manual loop to calculate the sum of squares, which is straightforward but can be inefficient for large datasets. The optimized version uses
#Improved version with edge case handling
def sum_of_squares_optimized(numbers):
    if not isinstance(numbers, list):
        raise TypeError("Input must be a list of numbers.")
    return sum(x * x for x in numbers if isinstance(x, (int, float)))
print(sum_of_squares_optimized(range(1000000)))  # Output will be the sum of squares from 0 to 999999
try:
    print(sum_of_squares_optimized("not a list"))  # This will raise an error
except TypeError as e:
    print(e)  # Output will be "Input must be a list of numbers."
try:    print(sum_of_squares_optimized([1, 2, "three", 4, 5]))  # This will raise an error
except TypeError as e:
    print(e)  # Output will be "Input must be a list of numbers."

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  POSTMAN CONSOLE
PS C:\Users\Sreevani B G\OneDrive\ドキュメント\AI> & "C:/Users/Sreevani B G/AppData/Local/Microsoft/WindowsApps/python3.13.exe" "c:/Users/Sreevani B G/OneDrive/ドキュメント/A
  File "c:\Users\Sreevani B G\OneDrive\ドキュメント\AI\lab10.py", line 387, in sum_of_squares_optimized
    raise TypeError("Input must be a list of numbers.")
TypeError: Input must be a list of numbers.
PS C:\Users\Sreevani B G\OneDrive\ドキュメント\AI>

Ln 28, Col 97  Spaces: 4  UTF-8  CRLF  { } Python  3.13.12 (Microsoft Store)  Python 3.13  Go Live
```